

Web Design & Databases

25/05/2025

Author: Rim Zino
Year: 2024/2025
Institution: SIST
Professor: Yassine Larbaoui
Module code: SEN4001
Student's ID: st20320120

Table of Contents

Site Map:.....3
Home Webpage:.....4
Registration webpage:.....9
A webpage to reset password:.....15
“Events-listing” webpage.....18
Webpage for Events-adding:.....19
Backend Database:.....21
Events table:.....21
Event registration table:.....21
Users table:.....22
Backend functions:.....22

Introduction

This report traces the advancement of a energetic web application outlined to assist students and chairmen oversee events and accounts inside an academic institution. The stage gives two role-based interfacing:

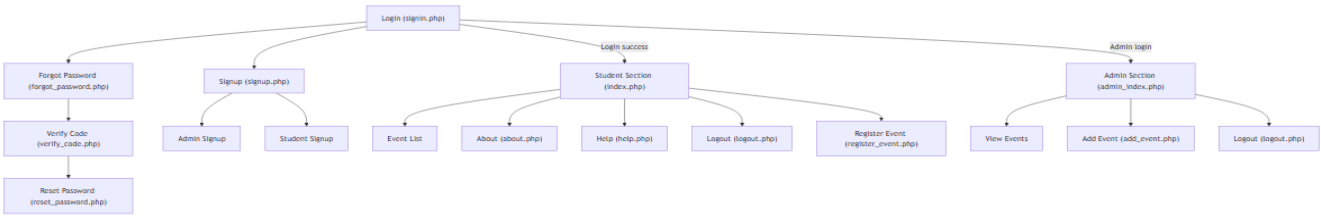
one for students to investigate and enlist for events, and another for admins to form and oversee those events. The most objective of the extend is to rearrange event communication and enrollment whereas keeping up a user-friendly and secure environment.

Students can effectively sign up, log in, browse up and coming events, and enroll with a single press. They moreover have get to to back pages like “About” and “Help.” Admins, on the other hand, can log in to a partitioned dashboard where they can include modern events, see all planned ones, and keep up control over event-related substance.

The application employments HTML, CSS, PHP, and MySQL to handle both the frontend interface and backend rationale. password recuperation, role-specific dashboards, and organized database structures guarantee that the application is utilitarian and secure.

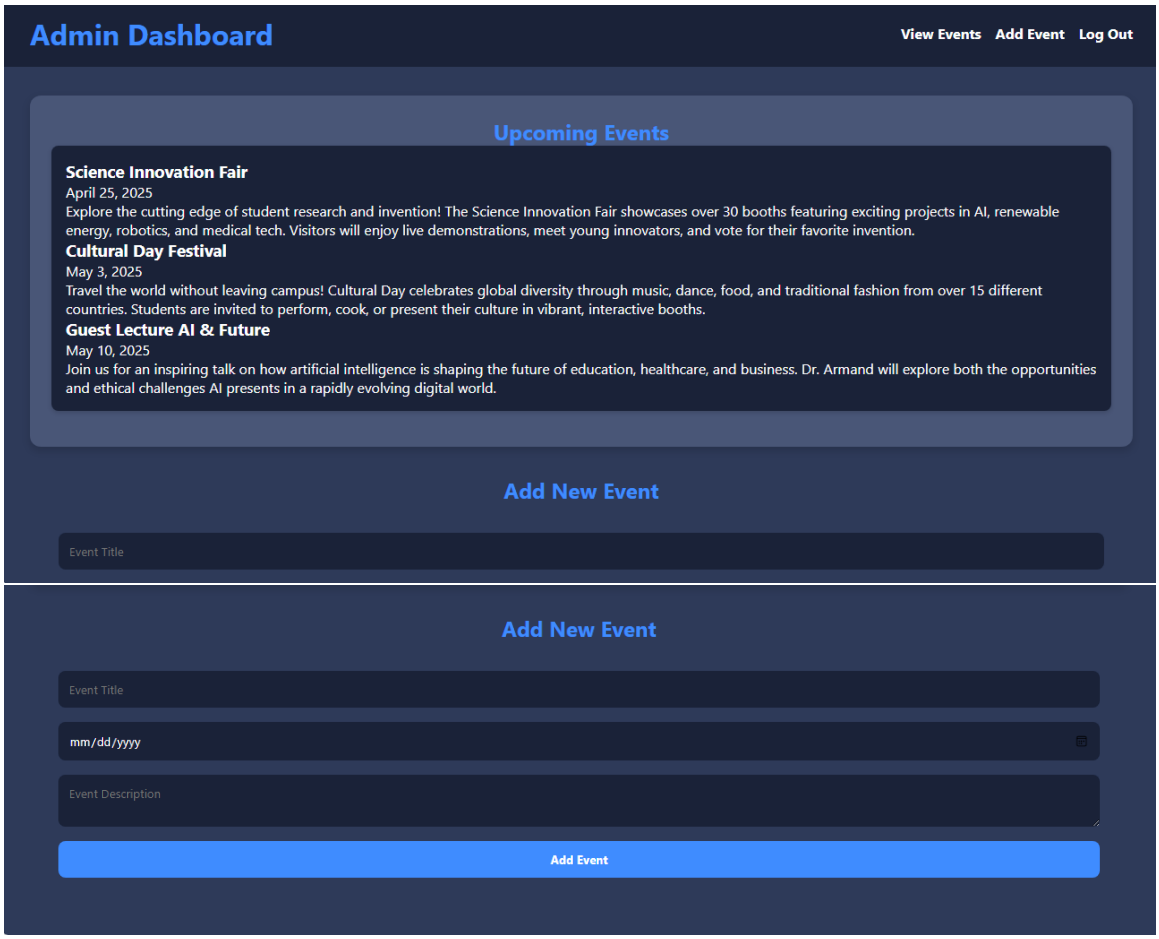
Generally, this framework exhibits how computerized apparatuses can streamline communication and engagement inside a school or college environment, whereas giving both students and admins a clear, effective way to remain associated and educated.

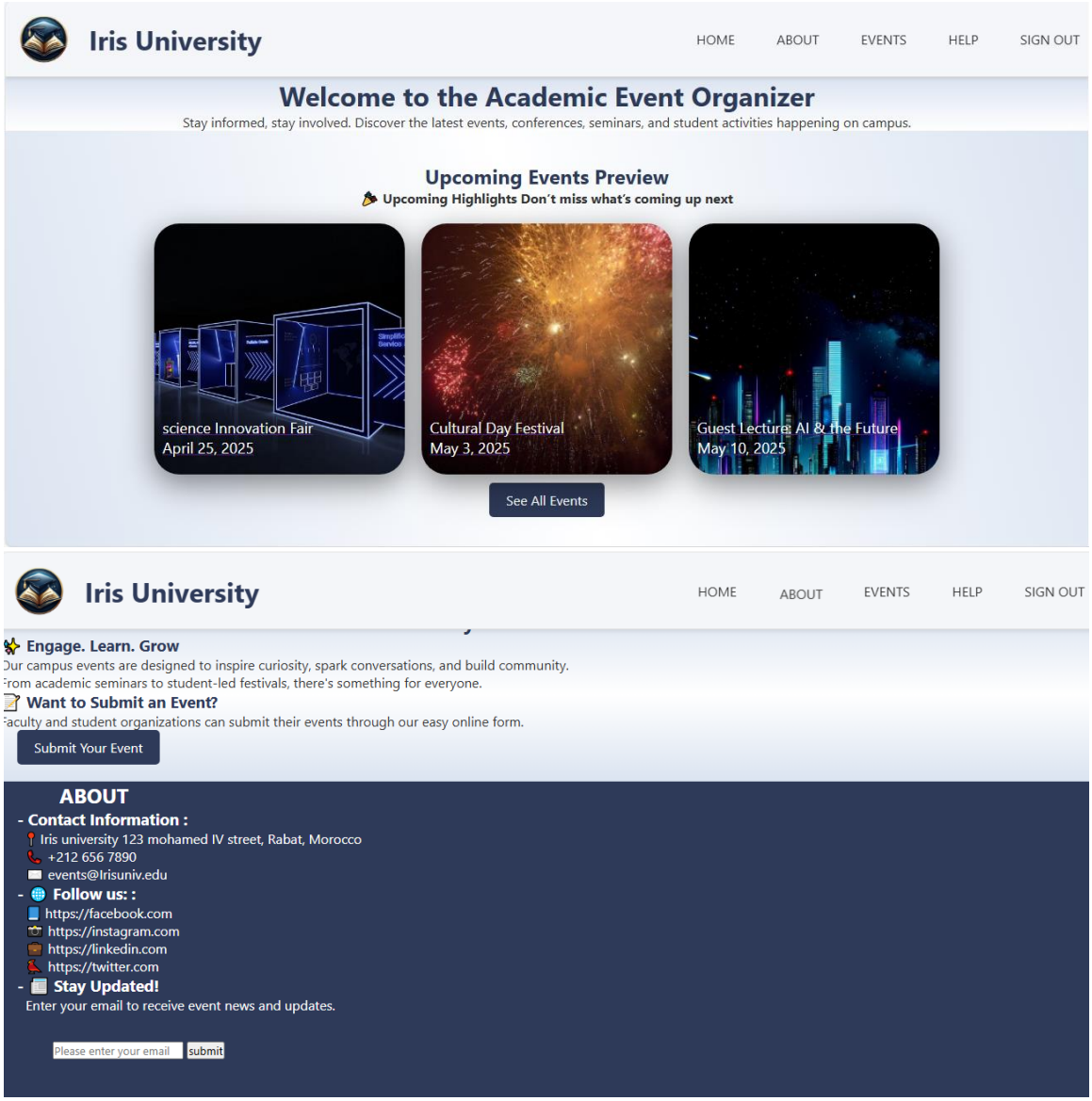
Site Map:



This sitemap lays out a straightforward and neighborly stream for a web app that makes a difference students and admins oversee their occasions and accounts without any cerebral pains. When somebody visits the location, the exceptionally to begin with thing they see is the Login page, where they can effectively sign in with their username and password. And in the event that they disregard their password? No stresses — they fair enter their e-mail, get a code to confirm it's truly them, and after that set a unused secret word. It's secure but super simple to take after. On the off chance that you're unused around here, the Signup page lets you make an account either as a understudy or an admin. This makes a difference keep things flawless and makes beyond any doubt everybody gets the correct get to and devices depending on who they are. Once logged in, understudies get taken to their possess dashboard, which has everything they require — a list of up and coming occasions, data around the stage, a few accommodating tips, and a logout button. When they discover an occasion they like, marking up is fair a tap absent. It's all almost making joining occasions speedy and hassle-free. For admins, there's a extraordinary dashboard fair for them where they can include modern occasions, see what's as of now planned, and keep things running easily behind the scenes. This way, everything remains organized and up to date. The total app guides clients actually from login to the proper put — understudies to their event-packed home, and admins to their control board. Resetting a watchword is straightforward and secure, so no one gets stuck. Within the conclusion, this sitemap is all almost making things simple and clear for everybody. It equalizations giving control to admins and keeping things basic for students — so everybody can do what they require without push. It's truly almost interfacing individuals to the proper occasions, keeping accounts secure, and making the total involvement smooth, neighborly, and inviting.

Home Webpage:





The Student page serves as the central center for understudies to effectively explore and lock in with the platform. It incorporates clear route joins to basic segments such as about, help, events, log out, guaranteeing understudies can rapidly get to data approximately the academic institution, discover support when required, or safely sign out.

Unmistakably shown on the student page are occasion cards exhibiting up and coming academic and extracurricular occasions. Each card gives key points of interest just like the occasion title, date, and a brief description, making it simple for students to find exercises that intrigued them. To energize support, each occasion card highlights a committed enlistment button. When clicked, this button coordinates understudies to a form where they can yield their data to connect the occasion.

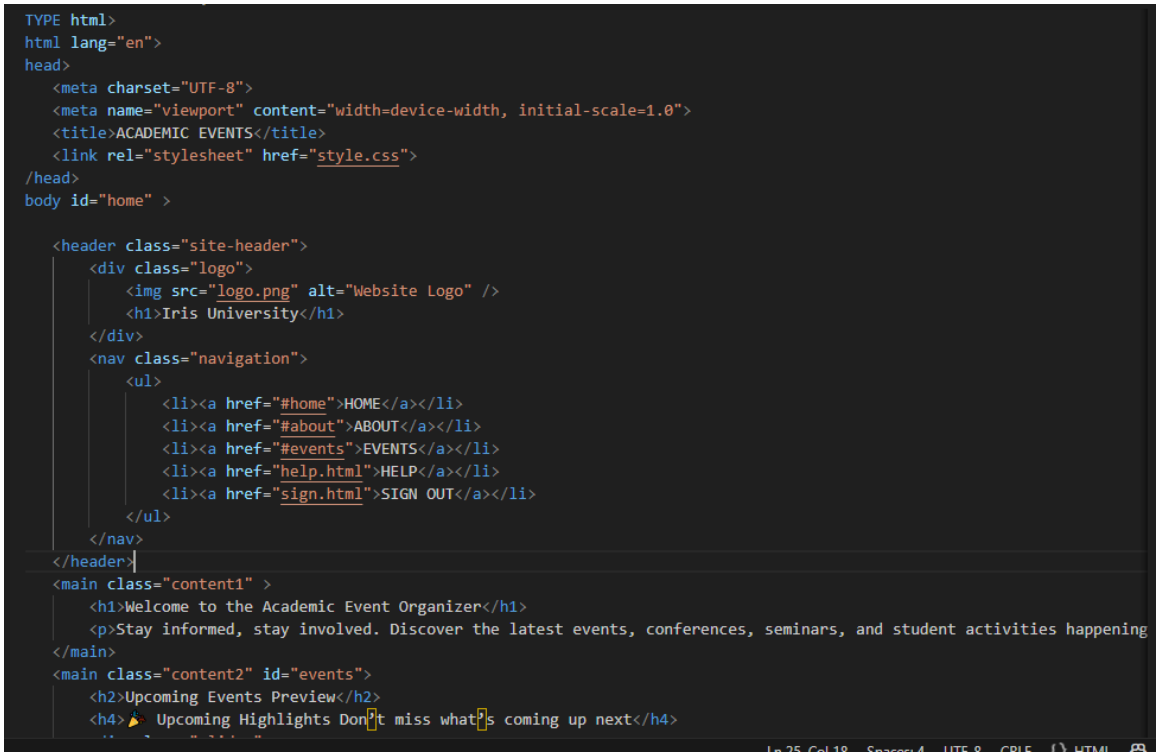
The plan centers on effortlessness and user-friendliness, permitting understudies to remain informed, get offer assistance, and effectively take an interest in campus life — all from one natural page.

The admin page is meant to give managers total control over properly running events. After logging in, admins are shown a thorough overview of all current events arranged on a card or list form. Every event shows relevant information including the event name, date, and description so enabling managers to quickly review upcoming events.

To add a new event, administrators only need to fill a basic form including fields for the event name, date, and a quick description. By saving the event straight to the database, clicking the "Add" button following data entry immediately updates the list of events. This seamless approach helps managers to easily keep a current and accurate event calendar.

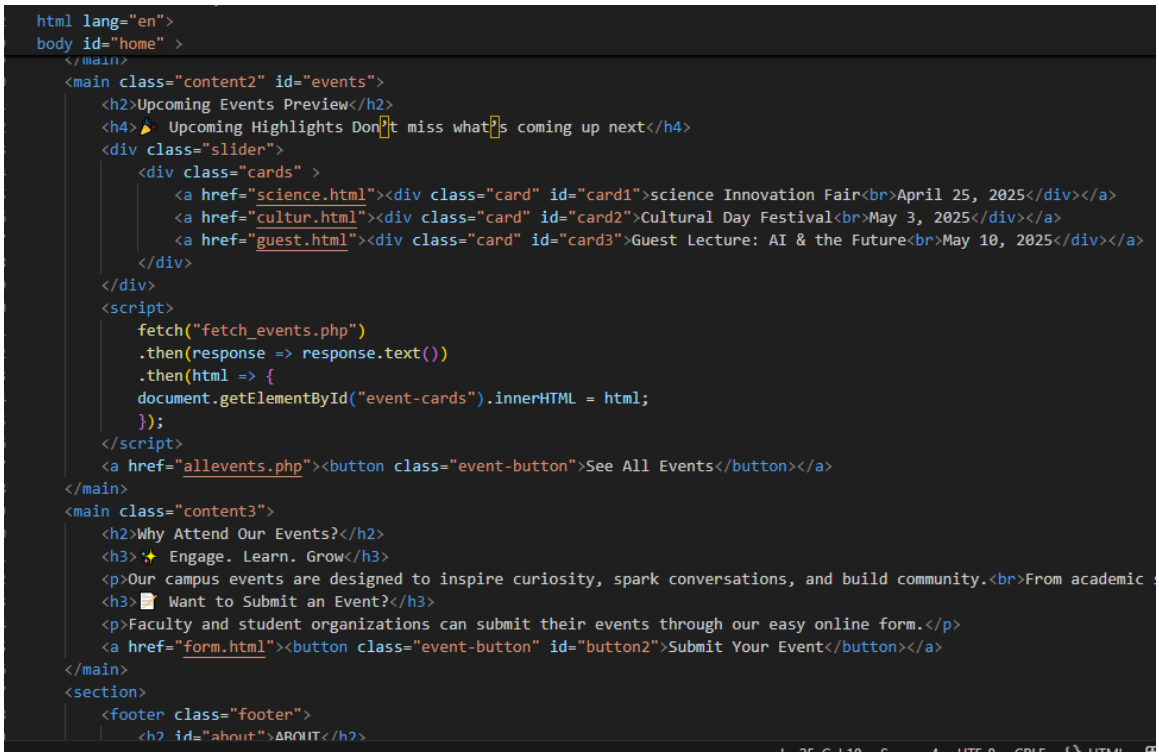
The page emphasizes usability and simplicity so that even people with little technical expertise may properly handle events. The administrative page supports seamless event management and helps keep a well-organized platform for every user by giving simple access to both viewing and adding events in one location.

And this is the essential parts of the code:



```
TYPE html>
html lang="en">
head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>ACADEMIC EVENTS</title>
   <link rel="stylesheet" href="style.css">
/head>
body id="home" >

   <header class="site-header">
      <div class="logo">
         <img src="logo.png" alt="Website Logo" />
         <h1>Iris University</h1>
      </div>
      <nav class="navigation">
         <ul>
            <li><a href="#home">HOME</a></li>
            <li><a href="#about">ABOUT</a></li>
            <li><a href="#events">EVENTS</a></li>
            <li><a href="help.html">HELP</a></li>
            <li><a href="sign.html">SIGN OUT</a></li>
         </ul>
      </nav>
   </header>
   <main class="content1" >
      <h1>Welcome to the Academic Event Organizer</h1>
      <p>Stay informed, stay involved. Discover the latest events, conferences, seminars, and student activities happening
   </main>
   <main class="content2" id="events">
      <h2>Upcoming Events Preview</h2>
      <h4> Upcoming Highlights Don't miss what's coming up next</h4>
```



```
<!DOCTYPE html>
<html lang="en">
<body id="home" >
    </main>
    <main class="content2" id="events">
        <h2>Upcoming Events Preview</h2>
        <h4>🎉 Upcoming Highlights Don't miss what's coming up next</h4>
        <div class="slider">
            <div class="cards" >
                <a href="science.html"><div class="card" id="card1">science Innovation Fair<br>April 25, 2025</div></a>
                <a href="cultur.html"><div class="card" id="card2">Cultural Day Festival<br>May 3, 2025</div></a>
                <a href="guest.html"><div class="card" id="card3">Guest Lecture: AI & the Future<br>May 10, 2025</div></a>
            </div>
        </div>
        <script>
            fetch("fetch_events.php")
            .then(response => response.text())
            .then(html => {
            document.getElementById("event-cards").innerHTML = html;
            });
        </script>
        <a href="allevents.php"><button class="event-button">See All Events</button></a>
    </main>
    <main class="content3">
        <h2>Why Attend Our Events?</h2>
        <h3>✨ Engage. Learn. Grow</h3>
        <p>Our campus events are designed to inspire curiosity, spark conversations, and build community.<br>From academic
        <h3>📝 Want to Submit an Event?</h3>
        <p>Faculty and student organizations can submit their events through our easy online form.</p>
        <a href="form.html"><button class="event-button" id="button2">Submit Your Event</button></a>
    </main>
    <section>
        <footer class="footer">
            <h2 id="about">ABOUT</h2>
```

Admin backend:

```php
<?php
session_start();

if (!isset($_SESSION['user_id']) || $_SESSION['user_role'] !== 'admin') {
    header("Location: admin.html");
    exit();
}

$conn = new mysqli("localhost", "root", "Reem1234@", "website");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $title = $conn->real_escape_string($_POST['title']);
    $description = $conn->real_escape_string($_POST['description']);
    $event_date = $_POST['event_date'];
    $created_by = $_SESSION['user_id'];

    $stmt = $conn->prepare("INSERT INTO events (title, description, event_date, created_by) VALUES (?, ?, ?, ?)");
    $stmt->bind_param("sssi", $title, $description, $event_date, $created_by);

    if ($stmt->execute()) {
        header("Location: admin_index.php?event_added=1");
        exit();
    } else {
        echo "Error adding event: " . $stmt->error;
    }

    $stmt->close();
```

admin_index.php

```php
    if ($_SERVER["REQUEST_METHOD"] === "POST") {
        $created_by = $_SESSION['user_id'];

        $stmt = $conn->prepare("INSERT INTO events (title, description, event_date, created_by) VALUES (?, ?, ?, ?)");
        $stmt->bind_param("sssi", $title, $description, $event_date, $created_by);

        if ($stmt->execute()) {
            header("Location: admin_index.php?event_added=1");
            exit();
        } else {
            echo "Error adding event: " . $stmt->error;
        }

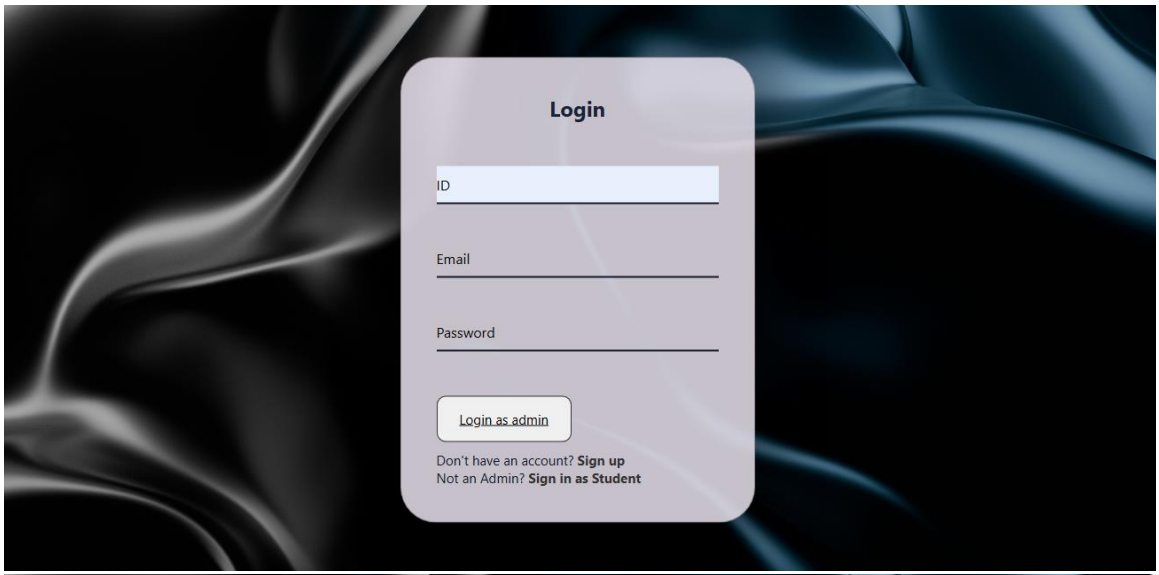
        $stmt->close();
    }
    ?>

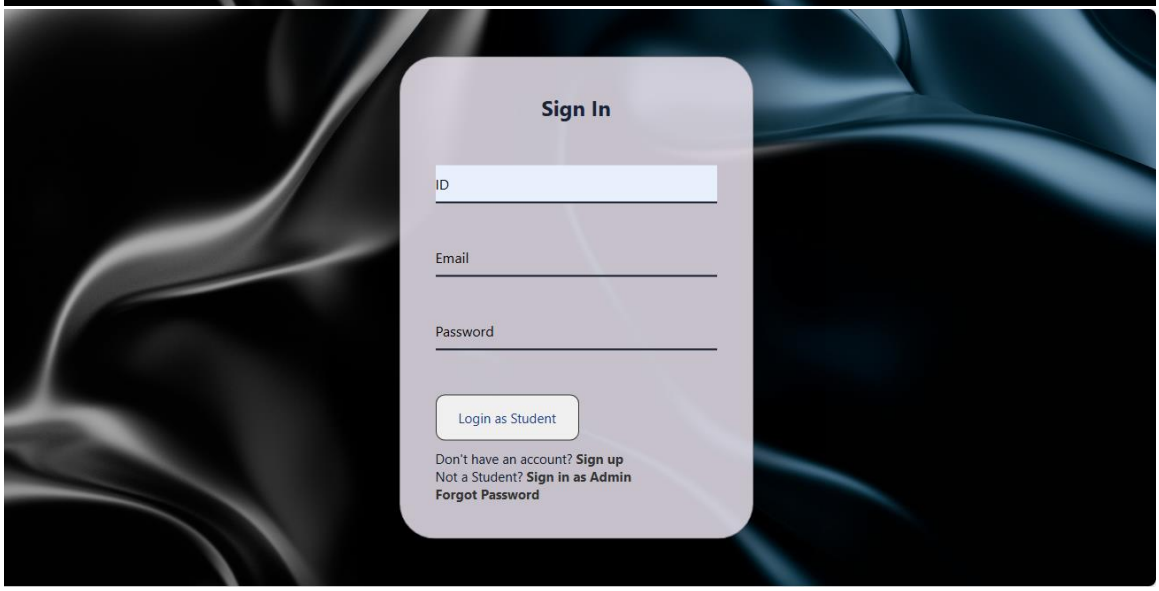
    <?php if (isset($_GET['event_added'])): ?>
        <p style="color: lightgreen;">Event added successfully!</p>
    <?php endif; ?>

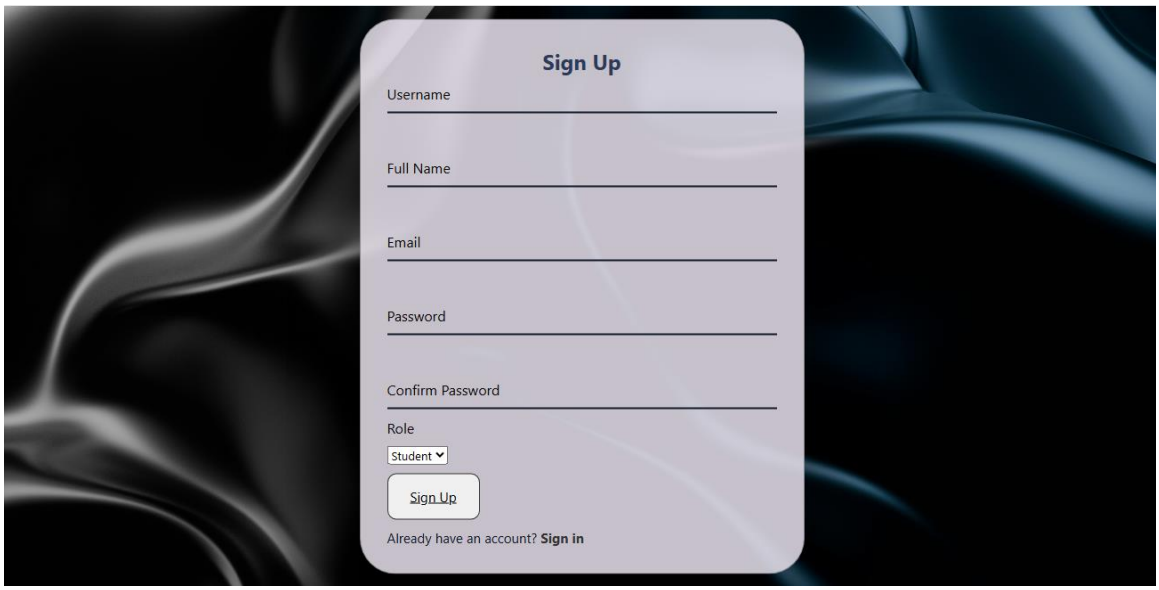
    <div class="main-content">
        <h2>All Events</h2>
        <?php
        $result = $conn->query("SELECT * FROM events ORDER BY event_date DESC");

        if ($result && $result->num_rows > 0) {
            while ($event = $result->fetch_assoc()) {
                echo "<div class='event-card'>";
                echo "<h3>" . htmlspecialchars($event['title']) . "</h3>";
                echo "<p><strong>Date:</strong> " . htmlspecialchars($event['event_date']) . "</p>";
                echo "<p>" . nl2br(htmlspecialchars($event['description'])) . "</p>";
                echo "</div>";
```

Registration webpage:


Login
ID
Email
Password
Login as admin
Don't have an account? Sign up
Not an Admin? Sign in as Student


Sign In
ID
Email
Password
Login as Student
Don't have an account? Sign up
Not a Student? Sign in as Admin
Forgot Password



The login webpage is the door to the energetic site, outlined to supply a smooth and secure get to point for both students and admins. It highlights a clean and inviting plan that facilitates easy sign-in or sign-up with a simple and direct.

The sign-in area permits users to log in utilizing their mail and password. This basic shape requires the user to enter their enrolled mail address and corresponding password to pick up get to. The framework at that point confirms these qualifications against the database, guaranteeing that as it were authorized users can continue. Both students and admins utilize this same sign-in page, keeping up consistency within the user experience. Be that as it may, the backend stores their data in an unexpected way to distinguish roles and control get to levels fittingly.

For new users, there is a clear sign-up choice accessible on the same page. The sign-up shape collects basic data counting username, full name, email, password, and password confirmation to guarantee exactness and security. Users must moreover select their role – either admin or student – which decides their consents and get to inside the framework. This part determination is basic since it guides the framework on how to handle user data and what parts of the site each user can get to.

The plan equalizations effortlessness with functionality, including user-friendly input areas, supportive placeholder content, and clear approval messages to guide users through the method. Security highlights such as password affirmation offer assistance minimize errors during registration, while role-based separation guarantees the site serves its shifted userbase viably. By and large, the login page acts as a secure and natural section point, inviting users and directing them easily into their personalized encounter.

And this is the essential code:

signin.php

```php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    if ($email && $password) {
        if ($result->num_rows === 1) {
            if (password_verify($password, $user['password'])) {
                exit();
            } else {
                $_SESSION['error_message'] = "Incorrect password.";
                header("Location: sign.php");
                exit();
            }
        } else {
            $_SESSION['error_message'] = "User not found.";
            header("Location: sign.php");
            exit();
        }

        $stmt->close();
    } else {
        $_SESSION['error_message'] = "Please fill all fields.";
        header("Location: sign.php");
        exit();
    }
} else {
    header("Location: index.html");
    exit();
}

$conn->close();
?>
```

signup.php

```php
<?php
session_start();

$conn = new mysqli("localhost", "root", "Reem1234@", "website");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Get and sanitize inputs
    $username = isset($_POST['username']) ? trim($_POST['username']) : '';
    $full_name = isset($_POST['full_name']) ? trim($_POST['full_name']) : '';
    $email = isset($_POST['email']) ? trim($_POST['email']) : '';
    $password = isset($_POST['password']) ? $_POST['password'] : '';
    $confirm = isset($_POST['confirm_password']) ? $_POST['confirm_password'] : '';
    $role = isset($_POST['role']) ? trim($_POST['role']) : '';

    // Check if passwords match
    if ($password !== $confirm) {
        echo "❌ Passwords do not match.";
        exit();
    }

    // Hash password
    $hashed_password = password_hash($password, PASSWORD_DEFAULT);

    // Check for existing email
    $checkStmt = $conn->prepare("SELECT user_id FROM users WHERE email = ?");
    $checkStmt->bind_param("s", $email);
    $checkStmt->execute();
    $checkResult = $checkStmt->get_result();
```

signin.php

```php
<?php
session_start();

$conn = new mysqli("localhost", "root", "Reem1234@", "website");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
if (isset($_SESSION['user_id'])) {
    header("Location: index.html");
    exit();}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $email = $_POST['email'] ?? '';
    $password = $_POST['password'] ?? '';

    if ($email && $password) {
        $stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");
        $stmt->bind_param("s", $email);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($result->num_rows === 1) {
            $user = $result->fetch_assoc();

            if (password_verify($password, $user['password'])) {
                $_SESSION['user_id'] = $user['user_id'];
                $_SESSION['user_email'] = $user['email'];
                $_SESSION['user_role'] = $user['role'];

                header("Location: form.html");
                exit();
            } else {
```

signup.php

```php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $checkStmt->bind_param("s", $email);
    $checkStmt->execute();
    $checkResult = $checkStmt->get_result();

    if ($checkResult->num_rows > 0) {
        echo "❌ An account with this email already exists.";
        exit();
    }

    // Insert user into database
    $stmt = $conn->prepare("INSERT INTO users (username, full_name, email, password, role) VALUES (?, ?, ?, ?, ?)");
    $stmt->bind_param("sssss", $username, $full_name, $email, $hashed_password, $role);

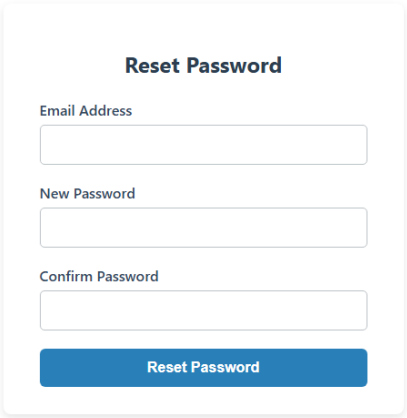
    if ($stmt->execute()) {
        $_SESSION['user_id'] = $stmt->insert_id;
        $_SESSION['user_email'] = $email;
        $_SESSION['user_role'] = $role;


        header("Location: index.html");
        exit();
    } else {
        echo "❌ Error signing up: " . $stmt->error;
    }

    $stmt->close();
    $checkStmt->close();
}

$conn->close();
?>
```

A webpage to reset password:



The reset password process is outlined to assist users recapture get to to their accounts rapidly and safely on the off chance that they disregard their password. From the login page, users can press a "Forgot Password" interface that takes them to a committed reset password page. Here, users enter their enrolled e-mail address to ask a password reset.

Once the user submits their e-mail, the framework creates a interesting confirmation code and sends it to the user's e-mail inbox. This additional step guarantees that as it were the legitimate proprietor of the mail can continue with resetting the password, adding an vital layer of security. After getting the code, users explore to a confirmation page where they enter the code to affirm their character.

Once the code is approved, the user is provoked to enter a new password and affirm it. This two-step secret word section makes a difference diminish mistakes and guarantees the new password is precisely what the user needs. After effectively upgrading their password, users can return to the login page to get to their account with the modern qualifications.

This prepare is outlined to be user-friendly and secure, adjusting ease of utilize with assurance against unauthorized get to. Clear enlightening and input direct users through each step, making beyond any doubt they never feel misplaced or befuddled amid password recuperation.

And this is the code:

forgot_password.php

```php
if ($conn->connect_error) {
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = $conn->real_escape_string($_POST['email']);

    // Check if email exists in users table
    $result = $conn->query("SELECT user_id FROM users WHERE email = '$email'");
    if ($result->num_rows == 0) {
        $error = "Email not found.";
    } else {
        // Generate reset code and expiration (15 minutes from now)
        $reset_code = rand(100000, 999999);
        $expires_at = date('Y-m-d H:i:s', strtotime('+15 minutes'));

        // Insert or update reset_code in reset_password table
        $conn->query("DELETE FROM users WHERE email = '$email'"); // remove old codes
        $stmt = $conn->prepare("INSERT INTO users(email, reset_password, expires_at) VALUES (?, ?, ?)");
        $stmt->bind_param("sis", $email, $reset_code, $expires_at);
        $stmt->execute();
        $stmt->close();

        // Send email (simplified, use your mail config)
        $subject = "Your Password Reset Code";
        $message = "Your password reset code is: $reset_code. It expires in 15 minutes.";
        $headers = "From: no-reply@example.com";

        mail($email, $subject, $message, $headers);

        $_SESSION['reset_email'] = $email;
        header("Location: verify_code.php");
        exit();
```

reset_password.php

```php
if (!isset($_SESSION['reset_email'], $_SESSION['code_verified']) || $_SESSION['code_verified'] !== true) {
    header("Location: forgot_password.php");
    exit();
}

$email = $_SESSION['reset_email'];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $password = $_POST['password'];
    $confirm_password = $_POST['confirm_password'];

    if ($password !== $confirm_password) {
        $error = "Passwords do not match.";
    } elseif (strlen($password) < 6) {
        $error = "Password should be at least 6 characters.";
    } else {
        $hashed_password = password_hash($password, PASSWORD_DEFAULT);

        // Update users table
        $stmt = $conn->prepare("UPDATE users SET password = ? WHERE email = ?");
        $stmt->bind_param("ss", $hashed_password, $email);
        if ($stmt->execute()) {
            // Clean up session and reset_password entry
            unset($_SESSION['reset_email'], $_SESSION['code_verified']);
            $conn->query("DELETE FROM reset_password WHERE email = '$email'");
            header("Location: signin.php?reset=success");
            exit();
        } else {
            $error = "Failed to update password.";
        }
        $stmt->close();
    }
```

verify_code.php

```php
<?php
session_start();
$conn = new mysqli("localhost", "root", "Reem1234@", "website");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if (!isset($_SESSION['reset_email'])) {
    header("Location: forgot_password.php");
    exit();
}

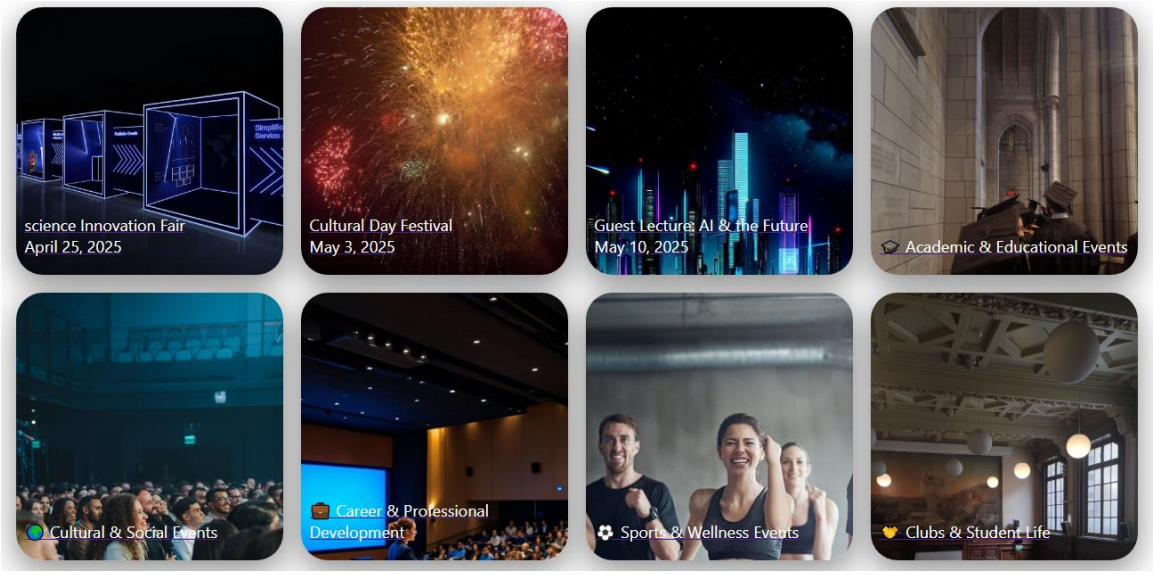
$email = $_SESSION['reset_email'];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $code = intval($_POST['code']);

    $stmt = $conn->prepare("SELECT * FROM reset_password WHERE email = ? AND reset_code = ? AND expires_at > NOW()");
    $stmt->bind_param("si", $email, $code);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows === 1) {
        $_SESSION['code_verified'] = true;
        header("Location: reset_password.php");
        exit();
    } else {
        $error = "Invalid or expired code.";
    }
    $stmt->close();
}
?>
```

“Events-listing” webpage



The Events-listing webpage serves as a comprehensive center where students can explore all the academic and extracurricular events organized by their institution. Open by implies of a committed button on the student residential page, this section allows students to viably browse through both up and coming and past events in one clear and organized see.

On passage, students are shown with a clean, user-friendly organize counting event cards or a list that joins basic focuses of intrigued such as the event title, date, zone, and a brief depiction. Each event segment is sketched out to quickly prompt students around what the event is, when and where it takes put, and why it might interested them. The arrange focuses to be ostensibly locks in be that as it may valuable, making a contrast students find vital information without pointless clutter.

Events are categorized or sorted by date, allowing students to see future openings for affiliation, as well as past events they might have missed or have to be review. This unquestionable perspective makes a contrast build a sense of community and coherence around campus works out.

The page may as well consolidate filtering or see choices, engaging students to contract down events by sort, date amplify, or academic division, updating their capacity to find events that alter with their interface or plans.

Also, each event card contains a enrollment button or interface for up and coming events, engaging basic back. This call to action is arranged to decrease boundaries, making it clear for students to sign up for events they find locks in.

In common, the Events-listing page plays a crucial portion in keeping students bolted in and taught, developing energetic participation in campus life, and making it basic to induce to all event-related information in one supportive put.

This is the essential part of the code:

allevents.php

```php
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>ACADEMIC EVENTS</title>
        <link rel="stylesheet" href="style.css">
    </head>
    <body>
        <div class="cards">
            <?php
                $conn = new mysqli("localhost", "root", "Reem1234@", "website");
                if ($conn->connect_error) {
                    die("Connection failed: " . $conn->connect_error);
                }

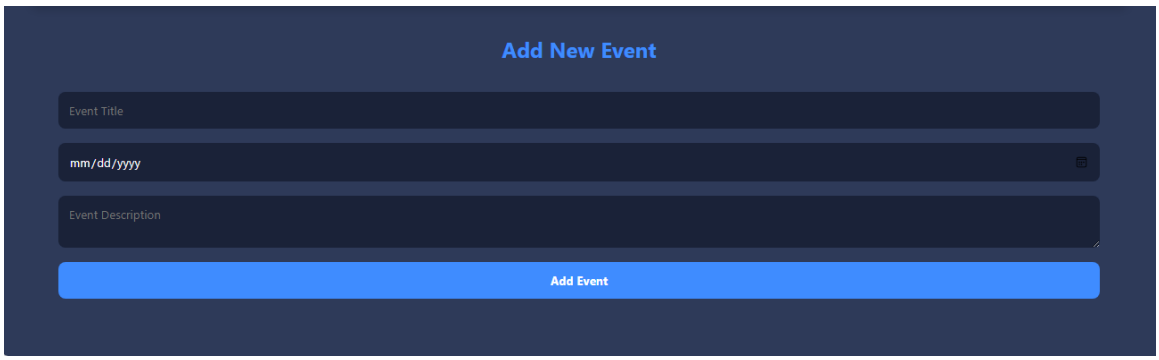
                $result = $conn->query("SELECT * FROM events ORDER BY event_date DESC");

                while ($row = $result->fetch_assoc()):
                ?>
                    <div class="card">
                        <h2><?= htmlspecialchars($row['title']) ?></h2>
                        <p><strong>Date:</strong> <?= htmlspecialchars($row['event_date']) ?></p>
                        <p><?= nl2br(htmlspecialchars($row['description'])) ?></p>
                    </div>
                <?php endwhile;

                $conn->close();
            ?>
            <a href="science.html"><div class="card" id="card1">science Innovation Fair<br>April 25, 2025</div></a>
            <a href="cultur.html"><div class="card" id="card2">Cultural Day Festival<br>May 3, 2025</div></a>
            <a href="guest.html"><div class="card" id="card3">Guest Lecture: AI & the Future<br>May 10, 2025</div></a>
```

The php part here let's the admin add the event on this page as I will explain now.

Webpage for Events-adding:



The add Events incorporate may be a key component of the admin dashboard, sketched out to give chiefs full control over supervising and organizing campus events capably. This convenience licenses admins to easily make cutting edge events by entering essential focuses of intrigued such as the event title, date, and a brief description, all through a coordinate and normal interface.

Upon logging in, admins can get to the Incorporate Events page, where a direct outline is standing by them. The shape consolidates regions for the event title, event date, and a portrayal box where they can donate critical information to bait understudy interest. Once all the inconspicuous components are filled inside, the admin clicks an “Add” button to surrender the event.

Behind the scenes, this settlement stores the unused event inside the database, making it right absent open for understudies to see on their household page and inside the comprehensive Events-listing portion. This steady affiliation between the admin input and student see ensures that event information is ceaselessly up-to-date and exact.

The arrange of the Incorporate Events page prioritizes ease of utilize, minimizing the time and effort required for admins to keep the event calendar current. It additionally makes a distinction maintain a strategic distance from goofs by endorsing inputs, such as guaranteeing the event date is considerable and the zones are not cleared out clear.

Besides, the admin dashboard frequently joins a range to see all as of now included events, allowing admins to overview, redesign, or delete events within the occasion that required. This nonstop organization capability ensures the event arrange remains organized and related.

By and large, the Incorporate Events highlight locks in admins to require charge of campus event coordination effortlessly, making past any question students are ceaselessly educated approximately up and coming openings to encourage included, learn, and interface.

This is the essential part of the code:

add_event.php

```php
<?php

$conn = new mysqli("localhost", "root", "Reem1234@", "website");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $title = $_POST['title'];
    $description = $_POST['description'];
    $event_date = $_POST['event_date'];

    // Insert into database
    $stmt = $conn->prepare("INSERT INTO events (title, description, event_date) VALUES (?, ?, ?)");
    $stmt->bind_param("sss", $title, $description, $event_date);

    if ($stmt->execute()) {
        header("Location: allevents.php");
        exit();
    } else {
        echo "Error: " . $stmt->error;
    }

    $stmt->close();
}

$conn->close();
?>
```

Backend Database:

EVENTS TABLE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	event_id	int			No	None		AUTO_INCREMENT	Change Drop More
2	title	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More
3	description	text	utf8mb4_0900_ai_ci		No	None			Change Drop More
4	event_date	date			No	None			Change Drop More
5	created_at	timestamp			Yes	CURRENT_TIMESTAMP		DEFAULT_GENERATED	Change Drop More

The events table stores all points of interest approximately the occasions overseen by the admin. Each occasion contains a interesting event_id which auto-increments to recognize each record. The title field holds the event's title with a restraint of 100 characters. The description field contains more point by point data approximately the event in content organize. The event_date stores the date on which the event is planned to happen. At last, the created_at field consequently records the timestamp when the event was included, making a difference track when each event was made.

EVENT REGISTRATION TABLE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int			No	None		AUTO_INCREMENT	Change Drop More
2	full_name	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More
3	email	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More
4	user_id	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More
5	event_name	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More

The event_registration table is outlined to store data around users who enroll for events. Each registration section is extraordinarily distinguished by an id, which is an auto-incremented essential key. The full_name and e-mail areas store the title and e-mail address of the member, guaranteeing clear recognizable proof. The user_id field records the interesting identifier of the student from the uers table, connecting the registration to a particular user. Finally, the event_name field captures the title of the event the student enlisted for, making a difference directors keep track of participation and cooperation for each event.

USERS TABLE:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	user_id	int			No	None		AUTO_INCREMENT	Change Drop More
2	username	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More
3	full_name	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More
4	email	varchar(100)	utf8mb4_0900_ai_ci		No	None			Change Drop More
5	password	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More
6	role	enum('student', 'admin')	utf8mb4_0900_ai_ci		No	None			Change Drop More
7	created_at	timestamp			Yes	CURRENT_TIMESTAMP		DEFAULT_GENERATED	Change Drop More
8	reset_password	varchar(10)	utf8mb4_0900_ai_ci		Yes	NULL			Change Drop More
9	expires_at	datetime			Yes	NULL			Change Drop More

The client table stores all enlisted users within the system. Each user incorporates a one of a kind user_id as the primary key. It incorporates areas such as username, full_name, mail, and password to oversee login credentials and user character. The part column indicates whether the user could be a student or an admin, guaranteeing appropriate get to control. The created_at field tracks account creation time. For password recuperation, reset_password stores the reset code and expires_at characterizes its legitimacy period. The e-mail field is recorded for quicker look operations, supporting both login and reset capacities safely and effectively.

Backend functions:

db.php

```php
<?php
$host = 'localhost';
$user = 'root';
$password = 'Reem1234@';
$database = 'website';

$conn = new mysqli($host, $user, $password, $database);

if ($conn->connect_error) {
    die("connection failed: " . $conn->connect_error);
}
?>
```

```php
$conn = new mysqli("localhost", "root", "Reem1234@", "website");
```

The backend of this web application is created utilizing PHP and serves as the basic bridge between the frontend user interfacing and the MySQL database. It handles all center functionalities such as login verification, event administration, enlistment preparing, and password recuperation — guaranteeing smooth, energetic intuitive for both students and admins.

One vital backend work is the “user confirmation” framework. When a student or admin submits their login accreditations, the backend employments PHP scripts to approve the submitted mail and password against the put away values within the 'user' table. On the off chance that coordinated, the client is allowed get to to the fitting dashboard based on their part ('student' or 'admin').

Another vital backend work is “occasion posting and enlistment”. The application brings all accessible occasions from the 'events' table utilizing SQL questions and shows them powerfully on the student dashboard. When a student registers for an event, their data (full title, e-mail, client ID, and chosen event) is safely embedded into the 'event_registration' table through PHP and MySQL integration.

Admins can “add new event” employing a shape on their dashboard. Upon accommodation, the backend gets the event title, portrayal, and date, at that point embeds the modern information into the 'events' table. These activities keep the database current and guarantee students continuously see the most recent event

The “reset password” handle is additionally overseen through PHP. When a user demands a password reset, the backend produces a special verification code, saves it in conjunction with an close timestamp within the 'user' table, and sends the code through e-mail. Once the user confirms the code, the backend permits them to upgrade their password securely.

In general, the backend capacities serve as the spine of the framework, guaranteeing information consistency, role-based route, secure operations, and real-time overhauls between clients and the database.

Conclusion

To entirety it all up, the net application is pointed at helping the organization of scholarly events and overseeing user accounts in a inviting and proficient manner. The event enrollment include within the student interface permits for event browsing and enrollment, whereas the admin

interface is custom-made towards proficient event administration and observing from the chairman viewpoint. The overarching objective is to move forward communication and coordination between the academic institution and its constituents.

Keyword resets empower privacy, whereas the signup handle gets significant information which is secured in a database concurring to organizational progression and part get to. The framework is well-structured and organized, making it user-friendly for people missing progressed innovative aptitudes.

Frontend user get to and backend get to functionalities are coordinates within the application through the utilize of HTML, CSS, PHP, and MySQL. The coherent course of action of interrelated information clusters inside the SQL database tables guarantees smooth upgrades and recovery, clearing the way for streamlined information dealing with and user operations.

The extend grandstands how having a web stage, no matter how essential, can ease the strain of event administration in other academic teach. With a few more work, alarms, portable versioning, and other changes arranged for long-standing time, the sys has incredible potential.

References

Welling, L. and Thomson, L., 2017. *PHP and MySQL web development*. 5th ed. Boston: Addison-Wesley.

Duckett, J., 2011. *HTML and CSS: Design and build websites*. Indianapolis: Wiley.

Ullman, L., 2017. *PHP for the web: Visual QuickStart guide*. 5th ed. San Francisco: Peachpit Press.

Beighley, L. and Morrison, M., 2009. *Head First PHP & MySQL*. 1st ed. Sebastopol: O'Reilly Media.

Nixon, R., 2018. *Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5*. 5th ed. Sebastopol: O'Reilly Media.

Oracle, 2024. *MySQL 8.0 Reference Manual*. [online] Available at: https://dev.mysql.com/doc/ [Accessed 25 May 2025].